FastBite

(notre application de livraison de repas, type Deliveroo)

TP6 — Interface graphique Swing et programmation événementielle

[Prénom NOM 1] · [Prénom NOM 2] · [Prénom NOM 3]

Programmation orientée objet en Java — ECE Paris, Ing2

Enseignant : J.-P. Segado · octobre 2026

Sommaire

1 **Q1 — Maquette (wireframes)**
les 3 fenêtres dessinées avant de coder (1 slide)

2 **Q2 — Storyboard**
la navigation entre les pages, les erreurs, les fichiers (1 slide)

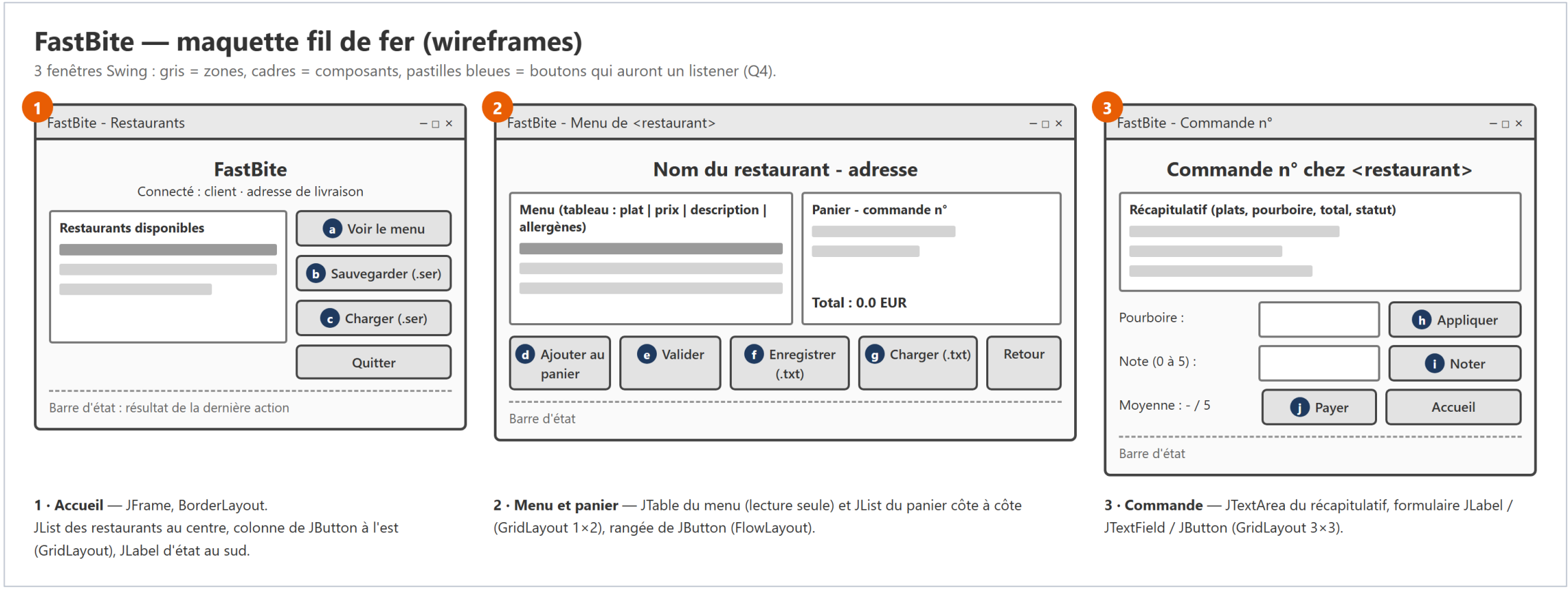
3 **Q3 — Fenêtres Swing**
code commenté et fenêtres obtenues (2 slides)

4 **Q4 — Listeners, exceptions, fichiers**
clics, erreurs affichées, .txt et .ser (3 slides)

5 **Bilan, sources et IA**
ce qui marche, limites, outils utilisés

Point de départ : le code du TP5 (classes métier, 9 exceptions, fichier texte, sérialisation). Le TP6 n'ajoute que des classes d'interface ; le métier n'est pas réécrit.

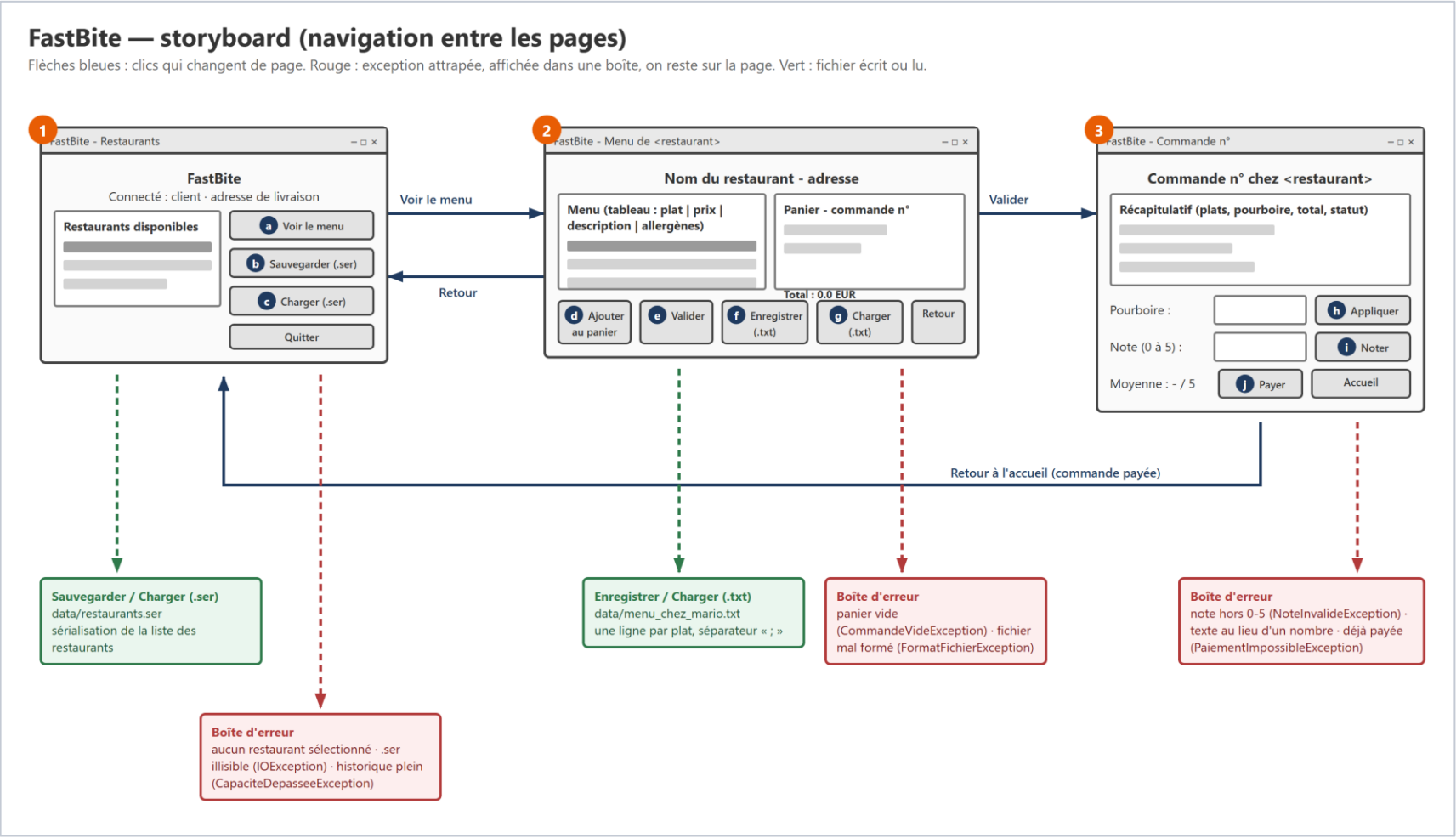
Q1 — Maquette fil de fer (wireframes)



Capture 1 — maquette des 3 fenêtres, générée par IA à partir de nos TP2 à TP5

- Pourquoi 3 fenêtres ? Elles suivent le cas d'utilisation principal du TP2 : choisir un restaurant, composer sa commande, payer et noter.
- Chaque zone correspond à un composant Swing (JList, JTable, JTextField, JButton) ; les pastilles bleues sont les boutons qui auront un listener.

Q2 — Storyboard : navigation entre les pages



Capture 2 — storyboard généré par IA

Lecture

- Bleu : un clic change de page (Voir le menu, Valider, Retour).
- Rouge : une exception est attrapée ; une boîte d'erreur s'ouvre et on reste sur la page.
- Vert : un fichier est écrit ou lu (.ser sur l'accueil, .txt sur le menu).

Choix

- Une seule fenêtre visible à la fois : l'accueil est caché, pas détruit, pour y revenir.
- Les fenêtres partagent un objet Donnees (restaurants, client).

Q3 — Construire une fenêtre Swing : le code

• FenetreAccueil.java

```
24  public class FenetreAccueil extends JFrame implements ActionListener {
```

• FenetreAccueil.java

```
50          // --- CENTRE : liste des restaurants (une seule selection a la fois)
51          listeRestaurants.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
52          listeRestaurants.setFont(new Font("SansSerif", Font.PLAIN, 16));
53          JScrollPane centre = new JScrollPane(listeRestaurants);
54          centre.setBorder(BorderFactory.createTitledBorder("Restaurants disponibles"));
55
56          // --- EST : colonne de boutons (GridLayout = cases de meme taille)
57          JPanel est = new JPanel(new GridLayout(4, 1, 0, 8));
58          est.add(boutonMenu);
59          est.add(boutonSauver);
60          est.add(boutonCharger);
61          est.add(boutonQuitter);
62
63          // --- SUD : barre d'etat (resultat de la derniere action)
64          etat.setBorder(BorderFactory.createEmptyBorder(6, 4, 2, 4));
65
66          // BorderLayout : 5 zones (NORTH, CENTER, EAST, SOUTH, WEST)
67          JPanel contenu = new JPanel(new BorderLayout(10, 10));
68          contenu.setBorder(BorderFactory.createEmptyBorder(12, 12, 8, 12));
69          contenu.add(nord, BorderLayout.NORTH);
70          contenu.add(centre, BorderLayout.CENTER);
71          contenu.add(est, BorderLayout.EAST);
72          contenu.add(etat, BorderLayout.SOUTH);
73          setContentPane(contenu);
74
75          // --- LISTENERS : la fenetre s'abonne aux clics des 4 boutons
76          boutonMenu.addActionListener(this);
77          boutonSauver.addActionListener(this);
78          boutonCharger.addActionListener(this);
79          boutonQuitter.addActionListener(this);
80
81          rafraichir();
82          setSize(640, 400);
83          setLocationRelativeTo(null);              // centre de l'ecran
84          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Capture 3 — FenetreAccueil : composants, layouts, abonnement aux boutons

• FastBiteGUI.java

```
19      public static void main(String[] args) {
20          installerApparence();
21          // Swing demande que les fenetres soient creees dans son propre fil (thread)
22          SwingUtilities.invokeLater(new Runnable() {
23              @Override
24              public void run() {
25                  Donnees donnees = new Donnees();
26                  FenetreAccueil accueil = new FenetreAccueil(donnees);
27                  try {
28                      donnees.creerJeuDEssai();
29                      accueil.rafraichir();
30                  } catch (CapaciteDepasseeException e) {
31                      Messages.erreur(accueil, "Demarrage", e.getMessage());
32                  }
33                  accueil.setVisible(true);
34              }
35          });
36      }
```

Capture 4 — FastBiteGUI.main : lancement

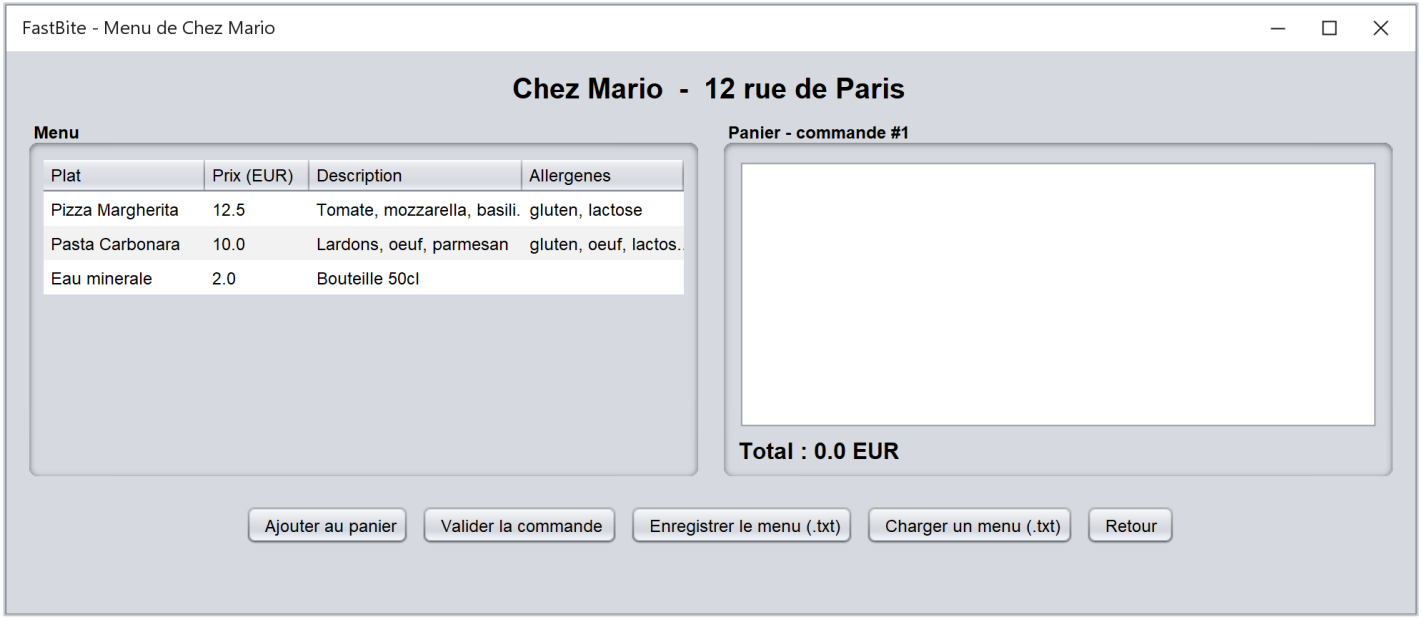
Ce que fait ce code

- La classe hérite de JFrame : c'est une fenêtre.
- BorderLayout : 5 zones (nord, centre, est, sud). GridLayout : cases égales pour la colonne de boutons.
- JScrollPane autour de la JList : barre de défilement si la liste est longue.
- addActionListener(this) : la fenêtre écoute ses boutons.
- invokeLater : Swing crée les fenêtres dans son propre fil d'exécution.

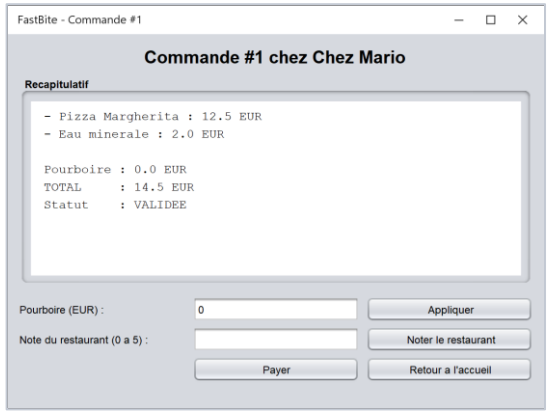
Q3 — Les 3 fenêtres obtenues



Capture 5 — page 1 : accueil (JList + boutons)



Capture 6 — page 2 : menu (JTable) et panier (JList)



Capture 7 — page 3 : commande

- Fidèle à la maquette : mêmes zones, mêmes boutons.
- JTable en lecture seule pour le menu : le tableau lit le vrai menu du restaurant (objets Plat du TP3).
- Apparence Nimbus : identique sur Windows, Mac et Linux.

Q4 — Listeners : un clic met à jour l'affichage

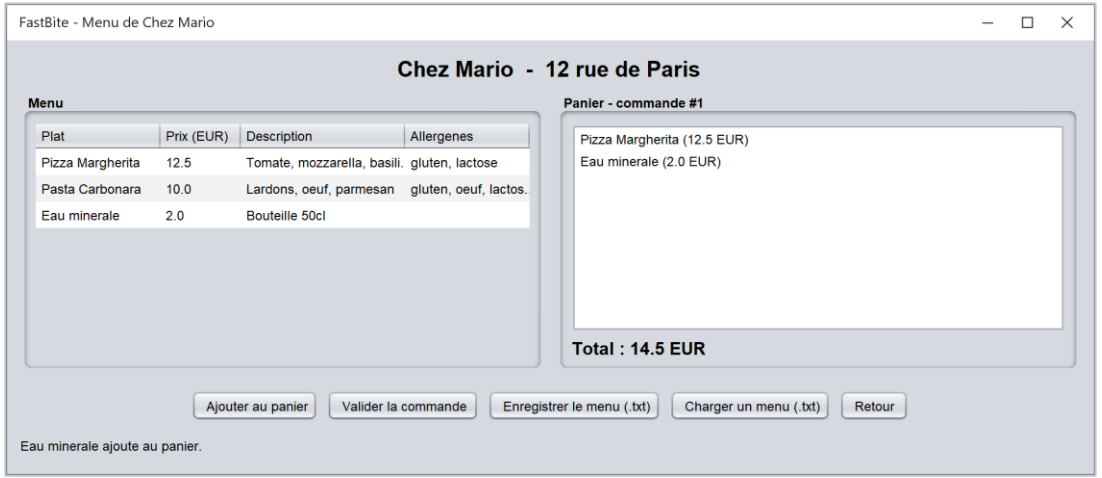
• FenetreAccueil.java

```java
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();
    if (source == boutonMenu) {
        ouvrirMenu();
    } else if (source == boutonSauver) {
        sauvegarder();
    } else if (source == boutonCharger) {
        charger();
    } else if (source == boutonQuitter) {
        dispose();
    }
}
```

• FenetreMenu.java

```java
private void ajouterAuPanier() {
    int ligne = tableMenu.getSelectedRow();
    if (ligne < 0) {
        Messages.erreur(this, "Aucun plat", "Selectionnez d'abord un plat dans le menu.");
        return;
    }
    try {
        commande.ajouterPlat(restaurant.getMenu()[ligne]);
        rafraichirPanier();                                  // mise a jour de l'affichage
        etat.setText(restaurant.getMenu()[ligne].getNom() + " ajoute au panier.");
    } catch (PlatIndisponibleException ex) {
        Messages.erreur(this, "Plat indisponible", ex.getMessage());
    }
}
```

Capture 8 — actionPerformed + getSource, puis le traitement d'un clic



Capture 9 — test : 2 clics « Ajouter au panier » → total 14.5 EUR

• FenetreCommande.java

```java
// --- LISTENERS (classes anonymes)
boutonPourboire.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) { appliquerPourboire(); }
});
boutonNoter.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) { noter(); }
```

Capture 10 — autre écriture : classe anonyme (page 3)

- Deux façons vues au cours 6 : la fenêtre implémente ActionListener (getSource dit quel bouton), ou une classe anonyme par bouton.
- Le listener modifie le modèle (commande.ajouterPlat) puis rafraîchit la vue.

Q4 — Les exceptions du TP5 dans l'interface

• FenetreMenu.java

```java
174    private void validerCommande() {
175        try {
176            commande.valider();
177            restaurant.recevoirCommande(commande);
178            FenetreCommande page = new FenetreCommande(accueil, donnees, commande);
179            page.setVisible(true);
180            dispose();                                    // on ferme la page 2
181        } catch (CommandeVideException ex) {
182            Messages.erreur(this, "Commande vide", ex.getMessage());
183        }
184    }
```

• FenetreCommande.java

```java
126    private void noter() {
127        Restaurant r = commande.getRestaurant();
128        try {
129            double note = Double.parseDouble(champNote.getText().trim().replace(',', '.'));
130            r.ajouterNote(note);
131            moyenne.setText("Moyenne : " + r.getNoteMoyenne() + " / 5");
132            etat.setText("Merci ! Note " + note + " enregistree pour " + r.getNom() + ".");
133        } catch (NumberFormatException ex) {
134            Messages.erreur(this, "Note invalide", "\"" + champNote.getText() + "\" n'est pas un nombre.");
135        } catch (NoteInvalideException ex) {
136            Messages.erreur(this, "Note invalide", ex.getMessage());
137        } catch (AucuneNoteException ex) {
138            moyenne.setText("Pas encore de note");
139        }
140    }
```

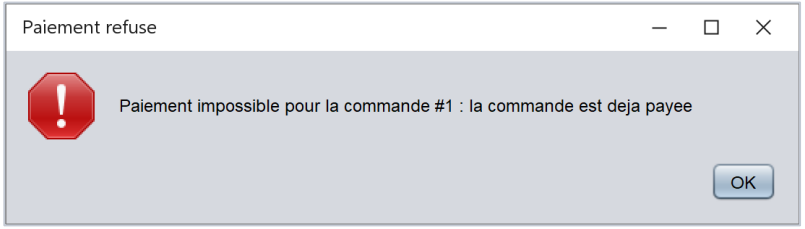
Capture 11 — try / catch dans les listeners : le message de l'exception va dans une boîte



Capture 12 — Valider avec un panier vide : CommandeVideException



Capture 13 — note 7 : NoteInvalideException



Capture 14 — 2e clic sur Payer : PaiementImpossibleException

- Les classes métier lèvent ou propagent (TP5) ; c'est le listener qui attrape et affiche.
- Après une erreur, la fenêtre reste utilisable et les données sont intactes.
- Nouveau cas : NumberFormatException si le champ ne contient pas un nombre.

Q4 — Fichier texte et sérialisation depuis l'interface

FenetreMenu.java

```java
public void chargerMenu(String fichier) {
    Restaurant relu = new Restaurant(restaurant.getNom(), restaurant.getAdresse(), restaurant.getZoneLivraison());
    try {
        relu.chargerMenu(fichier);
        donnees.getRestaurants().set(indice, relu);          // remplace l'ancien restaurant
        restaurant = relu;
        commande = new Commande(commande.getNumero(), donnees.getClient(), relu);   // panier vide
        remplirMenu();
        rafraichirPanier();
        etat.setText(relu.getNbPlats() + " plat(s) lu(s) depuis " + fichier);
    } catch (FormatFichierException ex) {
        Messages.erreur(this, "Fichier incorrect", ex.getMessage());
    } catch (IOException ex) {
        Messages.erreur(this, "Lecture impossible", ex.getMessage());
    } catch (CapaciteDepasseeException ex) {
        Messages.erreur(this, "Menu trop long", ex.getMessage());
    }
}
```

FenetreAccueil.java

```java
private void charger() {
    try {
        ArrayList<Restaurant> lus = Restaurant.chargerRestaurants(Donnees.FICHIER_SER);
        donnees.setRestaurants(lus);
        rafraichir();
        etat.setText(lus.size() + " restaurant(s) relu(s) depuis " + Donnees.FICHIER_SER);
    } catch (IOException ex) {
        // fichier absent, illisible ou qui n'est pas un flux d'objets Java
        Messages.erreur(this, "Chargement impossible", "Lecture de " + Donnees.FICHIER_SER + " : " + ex.getMessage());
    } catch (ClassNotFoundException ex) {
        Messages.erreur(this, "Chargement impossible", "Classe inconnue dans le fichier : " + ex.getMessage());
    }
}
```

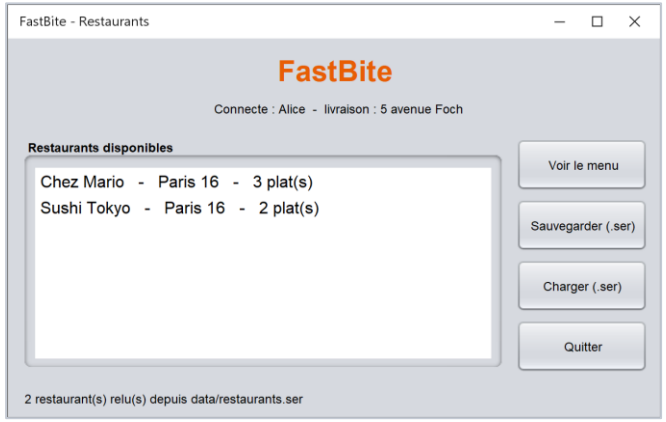
Capture 15 — chargerMenu (.txt) et charger (.ser) : appel des méthodes du TP5 + catch

data/menu_chez_mario.txt - Bloc-notes

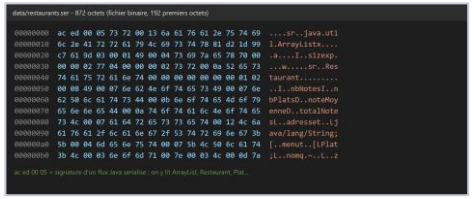
```
Pizza Margherita;12.5;Tomate, mozzarella, basilic;gluten,lactose
Pasta Carbonara;10.0;Lardons, oeuf, parmesan;gluten,oeuf,lactose
Eau minerale;2.0;Bouteille 50cl;
```

UTF-8 | 3 lignes | separateur : point-virgule

Capture 16 — data/menu_chez_mario.txt écrit par le bouton « Enregistrer »



Capture 17 — après « Charger (.ser) »



Capture 18 — le .ser (binaire)



Capture 19 — fichier .txt mal formé : FormatFichierException



Capture 20 — fichier .ser corrompu : IOException

- Le menu est lu dans un restaurant neuf : si le fichier est faux, l'écran n'est pas modifié.
- Test : liste vidée puis « Charger » → 2 restaurants, menus et note 4.5 retrouvés.

Bilan, sources et IA

Bilan

• Fait : maquette, storyboard, 3 fenêtres Swing reliées, 13 boutons avec listener, exceptions affichées, fichier .txt et .ser depuis l'interface.

• Vérifié : TestsTP6 simule les clics (doClick) : 27 vérifications passent ; les tests du TP5 passent toujours.

• Appris : JFrame et layouts, ActionListener (this ou classe anonyme), séparer l'interface du métier, le fil d'exécution de Swing.

• Bug trouvé grâce aux tests : après la lecture d'un .ser corrompu, le fichier restait ouvert (TP5). Corrigé avec deux ressources dans le try.

• Limites : un seul client, pas d'écran de connexion ni de livreur ; les images de fenêtres sont des rendus du programme de test.

• Améliorations : séparer en Modèle-Vue-Contrôleur (TP7), ajouter la page de suivi de livraison.

Sources

• J.-P. Segado, TP6 et cours 6 « Interface graphique Swing, AWT, Listeners » (ECE).

• Documentation Java SE (Oracle) : JFrame, ActionListener, ActionEvent, JOptionPane.

• Notre code du TP5 (exceptions, fichier, sérialisation).

IA utilisée

• Claude (Anthropic) : maquette, storyboard, code Swing, tests et slides générés par IA. Part d'aide de l'IA : à indiquer par l'équipe.

Prompt utilisé :

• « fais le tp6 dans la suite du tp5 fastbite » (avec le sujet du TP6 et notre code du TP5 fournis à l'IA)